Sprint -3 [Planner Habits Focus]

SPRINT 3 — DAILY PLANNER + HABITS + FOCUS COACH

Features 3, 14, and 9 — Full Implementation

PREREQUISITE: Sprint 1 + Sprint 2 must be complete

PASTE THIS INTO REPLIT AI AGENT

WHAT TO BUILD IN THIS SPRINT

1. Feature 3: AI Auto-Scheduler & Daily Planner (plan generation, timeline UI, morning/evening rituals, smart rescheduling)
2. Feature 14: AI Habit Tracker & Streak Engine (CRUD, AI scheduling, streaks, analytics)
3. Feature 9: AI Focus & Productivity Coach (focus sessions, focus protection, analytics, coaching)

After this sprint, verify:

- Navigate to "My Day" → see auto-generated daily plan with time blocks
- Drag blocks to reorder → saves changes
- Click "Start Focus" on a task → focus timer starts, notifications pause
- Create a habit "Deep Work 2h daily" → appears in daily plan

- Complete a habit → streak increments
- End of day → evening review shows completion stats

FEATURE 3: AI AUTO-SCHEDULER & DAILY PLANNER

3.1 tRPC Router

Create file: `src/server/routers/daily-plan.ts`

```
// src/server/routers/daily-plan.ts

import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";
import { logAiUsage } from "@/server/services/ai-usage-logge
r";

const timeBlockSchema = z.object({
  id: z.string(),
  type: z.enum(["task", "meeting", "focus", "break", "habit",
"communication", "planning", "buffer", "custom"]),
  title: z.string(),
  description: z.string().optional(),
  startTime: z.string(), // "09:00"
  endTime: z.string(),
  durationMinutes: z.number(),
  taskId: z.string().uuid().optional().nullable(),
  meetingId: z.string().uuid().optional().nullable(),
  habitId: z.string().uuid().optional().nullable(),
  priority: z.enum(["must_do", "should_do", "nice_to_do"]),
  energyLevel: z.enum(["high", "medium", "low"]),
  isFixed: z.boolean(),
  isCompleted: z.boolean().default(false),
  completedAt: z.string().optional().nullable(),
```

```
  isSkipped: z.boolean().default(false),
  actualDurationMinutes: z.number().optional().nullable(),
  scheduledByAI: z.boolean().default(false),
  aiReason: z.string().optional().nullable(),
});

export const dailyPlanRouter = router({
  /**
   * Get daily plan for a specific date
   */
  getByDate: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      date: z.string(), // "2026-03-04" ISO date string
    }))
    .query(async ({ input, ctx }) => {
      const plan = await prisma.dailyPlan.findUnique({
        where: {
          userId_date: {
            userId: ctx.user.id,
            date: new Date(input.date),
          },
        },
      });

      if (!plan) return null;
      return plan;
    }),

  /**
   * Generate AI daily plan for a specific date
   */
  generate: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      date: z.string(),
```

```
    }))
    .mutation(async ({ input, ctx }) => {
      const startTime = Date.now();
      const targetDate = new Date(input.date);

      // STEP 1: Gather all inputs for planning

      // Get user's scheduling preferences
      const prefs = await prisma.schedulingPreference.findUni
que({
        where: { userId: ctx.user.id },
      });
      const workingHours = (prefs?.workingHours as any) || {
start: "09:00", end: "18:00" };
      const lunchTime = (prefs?.lunchTime as any) || { start:
"12:30", end: "13:30" };
      const peakFocus = (prefs?.peakFocusWindow as any) || {
start: "10:00", end: "12:00" };

      // Get tasks due within 7 days assigned to this user
      const sevenDaysFromNow = new Date(targetDate);
      sevenDaysFromNow.setDate(sevenDaysFromNow.getDate() +
7);

      const tasks = await prisma.task.findMany({
        where: {
          assigneeId: ctx.user.id,
          workspaceId: input.workspaceId,
          completedAt: null,
          deletedAt: null,
          dueDate: { lte: sevenDaysFromNow },
        },
        orderBy: [{ dueDate: "asc" }, { priority: "desc" }],
        take: 20,
      });
```

```
      // Get today's calendar events (meetings)
      // Adjust this query to your actual Meeting/CalendarEve
nt model
      const meetings = await prisma.meeting.findMany({
        where: {
          participants: { some: { userId: ctx.user.id } },
          startTime: {
            gte: new Date(`${input.date}T00:00:00`),
            lt: new Date(`${input.date}T23:59:59`),
          },
        },
        orderBy: { startTime: "asc" },
      });

      // Get active habits
      const habits = await prisma.habit.findMany({
        where: {
          userId: ctx.user.id,
          isActive: true,
          deletedAt: null,
        },
      });

      // Get yesterday's incomplete tasks (carry-over)
      const yesterday = new Date(targetDate);
      yesterday.setDate(yesterday.getDate() - 1);
      const yesterdayPlan = await prisma.dailyPlan.findUnique
({
        where: { userId_date: { userId: ctx.user.id, date: ye
sterday } },
      });
      const carryOverBlocks = yesterdayPlan
        ? (yesterdayPlan.timeBlocks as any[]).filter((b: any)
=> !b.isCompleted && !b.isSkipped && b.taskId)
        : [];
```

```
      // Get duration estimates for tasks
      const taskIds = tasks.map(t => t.id);
      const estimates = await prisma.taskDurationEstimate.fin
dMany({
        where: { taskId: { in: taskIds } },
      });
      const estimateMap = new Map(estimates.map(e => [e.taskI
d, e.estimatedMinutes]));

      // STEP 2: Build the schedule with AI

      const tasksForAI = tasks.map(t => ({
        id: t.id,
        title: t.title,
        dueDate: t.dueDate?.toISOString().split("T")[0],
        priority: t.priority || "medium",
        estimatedMinutes: estimateMap.get(t.id) || 60,
        isOverdue: t.dueDate ? t.dueDate < targetDate : fals
e,
        isDueToday: t.dueDate?.toISOString().split("T")[0] ==
= input.date,
      }));

      const meetingsForAI = meetings.map(m => ({
        id: m.id,
        title: m.title,
        startTime: m.startTime.toTimeString().slice(0, 5),
        endTime: m.endTime.toTimeString().slice(0, 5),
        durationMinutes: Math.round((m.endTime.getTime() - m.
startTime.getTime()) / 60000),
      }));

      const habitsForAI = habits.map(h => ({
        id: h.id,
        name: h.name,
        durationMinutes: h.durationMinutes,
```

```
        preferredTimeRange: h.preferredTimeRange,
        priority: h.priority,
      }));

      const aiResponse = await aiService.generateJSON<{
        timeBlocks: Array<{
          type: string; title: string; startTime: string; end
Time: string;
          durationMinutes: number; taskId?: string; meetingI
d?: string; habitId?: string;
          priority: string; energyLevel: string; isFixed: boo
lean;
          aiReason: string;
        }>;
        summary: {
          totalPlannedMinutes: number; focusMinutes: number;
meetingMinutes: number;
          mustDoCount: number; shouldDoCount: number; niceToD
oCount: number;
          topPriority: string; aiReasoning: string; completio
nLikelihood: number;
        };
      }>(
        `Create an optimized daily schedule for ${input.dat
e}.

WORKING HOURS: ${workingHours.start} to ${workingHours.end}
LUNCH: ${lunchTime.start} to ${lunchTime.end}
PEAK FOCUS WINDOW: ${peakFocus.start} to ${peakFocus.end}
BUFFER BETWEEN MEETINGS: ${prefs?.bufferBetweenMeetings || 1
5} minutes

FIXED MEETINGS (cannot move):
${JSON.stringify(meetingsForAI, null, 2)}

TASKS TO SCHEDULE:
```

```
${JSON.stringify(tasksForAI, null, 2)}

HABITS TO INCLUDE:
${JSON.stringify(habitsForAI, null, 2)}

CARRY-OVER FROM YESTERDAY: ${carryOverBlocks.length} incomple
te tasks

RULES:
1. Place meetings first (they are fixed and cannot move)
2. Place MUST DO tasks (overdue + due today + high priority)
in peak focus window
3. Place habits in their preferred time ranges
4. Add 15-min breaks every 90 minutes
5. Add morning planning block (15 min at start of day)
6. Add evening review block (15 min at end of day)
7. Add communication blocks (30 min morning + 30 min after lu
nch)
8. Add buffer time between meetings
9. Leave at least 30 min unscheduled as buffer
10. Place SHOULD DO tasks in remaining good time slots
11. Place NICE TO DO tasks in lower-energy afternoon slots
12. Never overlap blocks
13. MUST DO = overdue + due today; SHOULD DO = due within 3 d
ays; NICE TO DO = rest

Return JSON with timeBlocks array sorted by startTime and a s
ummary object.
Each block needs: type, title, startTime (HH:MM), endTime (H
H:MM), durationMinutes, taskId/meetingId/habitId if applicabl
e, priority (must_do/should_do/nice_to_do), energyLevel (hig
h/medium/low), isFixed (true for meetings), aiReason.`,
        {
          systemPrompt: "You are an expert productivity sched
uler. Create optimal daily schedules that maximize deep work
time and respect human energy patterns. Return ONLY valid JSO
```

```
N.",
          temperature: 0.3,
        }
      );

      // STEP 3: Save the plan

      const timeBlocks = aiResponse.timeBlocks.map((block, in
dex) => ({
        ...block,
        id: crypto.randomUUID(),
        isCompleted: false,
        isSkipped: false,
        scheduledByAI: true,
      }));

      const plan = await prisma.dailyPlan.upsert({
        where: { userId_date: { userId: ctx.user.id, date: ta
rgetDate } },
        create: {
          userId: ctx.user.id,
          workspaceId: input.workspaceId,
          date: targetDate,
          status: prefs?.planMode === "automatic" ? "active"
: "generated",
          timeBlocks: timeBlocks as any,
          aiSummary: aiResponse.summary as any,
        },
        update: {
          timeBlocks: timeBlocks as any,
          aiSummary: aiResponse.summary as any,
          status: "generated",
        },
      });

      await logAiUsage({
```

```
        userId: ctx.user.id,
        workspaceId: input.workspaceId,
        feature: "planner",
        action: "generate_plan",
        model: "gpt-4o",
        inputTokens: 0, outputTokens: 0,
        durationMs: Date.now() - startTime,
      });

      return plan;
    }),

  /**
   * Update time blocks (user dragged, completed, skipped, et
c.)
   */
  updateBlocks: protectedProcedure
    .input(z.object({
      planId: z.string().uuid(),
      timeBlocks: z.array(timeBlockSchema),
    }))
    .mutation(async ({ input }) => {
      return prisma.dailyPlan.update({
        where: { id: input.planId },
        data: {
          timeBlocks: input.timeBlocks as any,
          userModifications: { increment: 1 },
        },
      });
    }),

  /**
   * Complete a time block
   */
  completeBlock: protectedProcedure
    .input(z.object({
```

```
      planId: z.string().uuid(),
      blockId: z.string(),
      actualDurationMinutes: z.number().optional(),
    }))
    .mutation(async ({ input }) => {
      const plan = await prisma.dailyPlan.findUnique({ where:
{ id: input.planId } });
      if (!plan) throw new Error("Plan not found");

      const blocks = plan.timeBlocks as any[];
      const blockIndex = blocks.findIndex((b: any) => b.id ==
= input.blockId);
      if (blockIndex === -1) throw new Error("Block not foun
d");

      blocks[blockIndex].isCompleted = true;
      blocks[blockIndex].completedAt = new Date().toISOString
();
      blocks[blockIndex].actualDurationMinutes = input.actual
DurationMinutes;

      // Calculate completion rate
      const totalBlocks = blocks.filter((b: any) => b.type !=
= "break" && b.type !== "buffer");
      const completedBlocks = totalBlocks.filter((b: any) =>
b.isCompleted);
      const completionRate = completedBlocks.length / totalBl
ocks.length;

      return prisma.dailyPlan.update({
        where: { id: input.planId },
        data: { timeBlocks: blocks, completionRate },
      });
    }),

  /**
```

```
   * Skip a time block
   */
  skipBlock: protectedProcedure
    .input(z.object({ planId: z.string().uuid(), blockId: z.s
tring() }))
    .mutation(async ({ input }) => {
      const plan = await prisma.dailyPlan.findUnique({ where:
{ id: input.planId } });
      if (!plan) throw new Error("Plan not found");

      const blocks = plan.timeBlocks as any[];
      const blockIndex = blocks.findIndex((b: any) => b.id ==
= input.blockId);
      if (blockIndex === -1) throw new Error("Block not foun
d");

      blocks[blockIndex].isSkipped = true;

      return prisma.dailyPlan.update({
        where: { id: input.planId },
        data: { timeBlocks: blocks },
      });
    }),

  /**
   * Save evening review note
   */
  saveReview: protectedProcedure
    .input(z.object({
      planId: z.string().uuid(),
      reviewNote: z.string().optional(),
    }))
    .mutation(async ({ input }) => {
      return prisma.dailyPlan.update({
        where: { id: input.planId },
        data: { reviewNote: input.reviewNote, status: "comple
```

```
ted" },
      });
    }),

  /**
   * Get or create scheduling preferences
   */
  getPreferences: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input, ctx }) => {
      return prisma.schedulingPreference.upsert({
        where: { userId: ctx.user.id },
        create: { userId: ctx.user.id, workspaceId: input.wor
kspaceId },
        update: {},
      });
    }),

  /**
   * Update scheduling preferences
   */
  updatePreferences: protectedProcedure
    .input(z.object({
      workingHours: z.any().optional(),
      timezone: z.string().optional(),
      workDays: z.array(z.number()).optional(),
      lunchTime: z.any().optional(),
      peakFocusWindow: z.any().optional(),
      preferMeetingsIn: z.string().optional(),
      minFocusBlockMinutes: z.number().optional(),
      bufferBetweenMeetings: z.number().optional(),
      autoScheduleEnabled: z.boolean().optional(),
      morningPlanningEnabled: z.boolean().optional(),
      morningPlanningTime: z.string().optional(),
      eveningReviewEnabled: z.boolean().optional(),
      eveningReviewTime: z.string().optional(),
```

```
      planMode: z.string().optional(),
    }))
    .mutation(async ({ input, ctx }) => {
      return prisma.schedulingPreference.update({
        where: { userId: ctx.user.id },
        data: input,
      });
    }),
});
```

3.2 Daily Plan Timeline UI

Create file: `src/components/daily-plan/plan-timeline.tsx`

```
// src/components/daily-plan/plan-timeline.tsx
// Main daily planner view with draggable time blocks

"use client";

import { useState, useCallback } from "react";
import { ChevronLeft, ChevronRight, RefreshCw, Settings, Pla
y, Check, SkipForward, Clock } from "lucide-react";
import { PlanBlock } from "./plan-block";
import { trpc } from "@/lib/trpc-client";
import { format, addDays, subDays, isToday } from "date-fns";

interface PlanTimelineProps {
  workspaceId: string;
}

export function PlanTimeline({ workspaceId }: PlanTimelinePro
ps) {
  const [selectedDate, setSelectedDate] = useState(new Date
());
  const dateStr = format(selectedDate, "yyyy-MM-dd");
```

```
  const planQuery = trpc.dailyPlan.getByDate.useQuery({ works
paceId, date: dateStr });
  const generateMutation = trpc.dailyPlan.generate.useMutatio
n({
    onSuccess: () => planQuery.refetch(),
  });
  const completeBlockMutation = trpc.dailyPlan.completeBlock.
useMutation({
    onSuccess: () => planQuery.refetch(),
  });
  const skipBlockMutation = trpc.dailyPlan.skipBlock.useMutat
ion({
    onSuccess: () => planQuery.refetch(),
  });

  const plan = planQuery.data;
  const timeBlocks = (plan?.timeBlocks as any[]) || [];
  const aiSummary = plan?.aiSummary as any;

  const handleGenerate = () => {
    generateMutation.mutate({ workspaceId, date: dateStr });
  };

  const handleComplete = (blockId: string) => {
    if (!plan) return;
    completeBlockMutation.mutate({ planId: plan.id, blockId
});
  };

  const handleSkip = (blockId: string) => {
    if (!plan) return;
    skipBlockMutation.mutate({ planId: plan.id, blockId });
  };

  return (
    <div className="max-w-2xl mx-auto p-4">
```

```
      {/* Header */}
      <div className="flex items-center justify-between mb-
4">
        <div className="flex items-center gap-2">
          <h1 className="text-xl font-semibold">
            📅 My Day — {format(selectedDate, "EEEE, MMMM
d")}
          </h1>
        </div>
        <div className="flex items-center gap-2">
          <button onClick={() => setSelectedDate(subDays(sele
ctedDate, 1))} className="p-1 hover:bg-gray-100 rounded">
            <ChevronLeft className="w-4 h-4" />
          </button>
          <button
            onClick={() => setSelectedDate(new Date())}
            className={`px-2 py-1 text-xs rounded ${isToday(s
electedDate) ? "bg-blue-100 text-blue-700" : "bg-gray-100"}`}
          >
            Today
          </button>
          <button onClick={() => setSelectedDate(addDays(sele
ctedDate, 1))} className="p-1 hover:bg-gray-100 rounded">
            <ChevronRight className="w-4 h-4" />
          </button>
          <button
            onClick={handleGenerate}
            disabled={generateMutation.isPending}
            className="flex items-center gap-1 px-3 py-1 text
-sm bg-purple-600 text-white rounded hover:bg-purple-700 disa
bled:opacity-50"
          >
            <RefreshCw className={`w-3 h-3 ${generateMutatio
n.isPending ? "animate-spin" : ""}`} />
            {plan ? "Regenerate" : "Generate Plan"}
          </button>
```

```
        </div>
      </div>

      {/* AI Briefing */}
      {aiSummary && (
        <div className="bg-purple-50 border border-purple-200
rounded-lg p-3 mb-4">
          <div className="flex items-center gap-1 text-xs tex
t-purple-600 font-medium mb-1">
            🤖 AI Briefing
          </div>
          <p className="text-sm text-gray-800">{aiSummary.aiR
easoning}</p>
          <p className="text-xs text-gray-500 mt-1">
            Estimated completion: {aiSummary.completionLikeli
hood}% likely
          </p>
        </div>
      )}

      {/* Time Blocks */}
      {planQuery.isLoading && (
        <div className="text-center py-12 text-gray-500">Load
ing your plan...</div>
      )}

      {!plan && !planQuery.isLoading && (
        <div className="text-center py-12">
          <p className="text-gray-500 mb-4">No plan for this
day yet.</p>
          <button
            onClick={handleGenerate}
            disabled={generateMutation.isPending}
            className="px-4 py-2 bg-purple-600 text-white rou
nded-lg hover:bg-purple-700"
          >
```

```
            🤖 Generate AI Plan
          </button>
        </div>
      )}

      {timeBlocks.length > 0 && (
        <div className="space-y-2">
          {timeBlocks.map((block: any) => (
            <PlanBlock
              key={block.id}
              block={block}
              onComplete={() => handleComplete(block.id)}
              onSkip={() => handleSkip(block.id)}
              onStartFocus={() => {
                // Navigate to focus session (Feature 9)
                // Will be connected when Feature 9 is built
              }}
            />
          ))}
        </div>
      )}

      {/* Summary Bar */}
      {aiSummary && (
        <div className="mt-4 bg-gray-50 border rounded-lg p-3
flex items-center justify-between text-sm">
          <div className="flex items-center gap-4">
            <span>
              Must: {timeBlocks.filter((b: any) => b.priority
=== "must_do" && b.isCompleted).length}/
              {timeBlocks.filter((b: any) => b.priority ===
"must_do").length} ✅
            </span>
            <span>
              Should: {timeBlocks.filter((b: any) => b.priori
ty === "should_do" && b.isCompleted).length}/
```

```
              {timeBlocks.filter((b: any) => b.priority ===
"should_do").length}
            </span>
            <span>
              Nice: {timeBlocks.filter((b: any) => b.priority
=== "nice_to_do" && b.isCompleted).length}/
              {timeBlocks.filter((b: any) => b.priority ===
"nice_to_do").length}
            </span>
          </div>
          <span className="text-gray-500">
            Focus: {aiSummary.focusMinutes}min | Meetings: {a
iSummary.meetingMinutes}min
          </span>
        </div>
      )}
    </div>
  );
}
```

Create file: `src/components/daily-plan/plan-block.tsx`

```
// src/components/daily-plan/plan-block.tsx

"use client";

import { Check, SkipForward, Play, Clock, GripVertical } from
"lucide-react";

interface PlanBlockProps {
  block: {
    id: string; type: string; title: string; startTime: strin
g; endTime: string;
    durationMinutes: number; priority: string; energyLevel: s
tring;
    isFixed: boolean; isCompleted: boolean; isSkipped: boolea
```

```
n; aiReason?: string;
  };
  onComplete: () => void;
  onSkip: () => void;
  onStartFocus: () => void;
}

const TYPE_STYLES: Record<string, { bg: string; icon: string
}> = {
  task: { bg: "border-l-4 border-l-blue-500", icon: "📋" },
  meeting: { bg: "border-l-4 border-l-purple-500 bg-purple-5
0", icon: "🟪" },
  focus: { bg: "border-l-4 border-l-red-500", icon: "🎯" },
  break: { bg: "border-l-4 border-l-green-500 bg-green-50", i
con: "☕" },
  habit: { bg: "border-l-4 border-l-orange-500", icon: "🔥"
},
  communication: { bg: "border-l-4 border-l-yellow-500", ico
n: "📧" },
  planning: { bg: "border-l-4 border-l-indigo-500 bg-indigo-5
0", icon: "🌅" },
  buffer: { bg: "border-l-4 border-l-gray-300 bg-gray-50", ic
on: "⏳" },
  custom: { bg: "border-l-4 border-l-teal-500", icon: "✏" },
};

const PRIORITY_BADGES: Record<string, string> = {
  must_do: "bg-red-100 text-red-700",
  should_do: "bg-yellow-100 text-yellow-700",
  nice_to_do: "bg-blue-100 text-blue-700",
};

export function PlanBlock({ block, onComplete, onSkip, onStar
tFocus }: PlanBlockProps) {
  const style = TYPE_STYLES[block.type] || TYPE_STYLES.custo
m;
```

```
  return (
    <div
      className={`rounded-lg p-3 ${style.bg} ${
        block.isCompleted ? "opacity-50 line-through" : ""
      } ${block.isSkipped ? "opacity-30" : ""} bg-white shado
w-sm hover:shadow-md transition-shadow`}
    >
      <div className="flex items-start gap-2">
        <GripVertical className="w-4 h-4 text-gray-300 cursor
-grab mt-0.5 flex-shrink-0" />
        <div className="flex-1 min-w-0">
          <div className="flex items-center gap-2">
            <span className="text-xs text-gray-500">{block.st
artTime} – {block.endTime}</span>
            <span className="text-xs">{style.icon}</span>
            <span className="font-medium text-sm truncate">{b
lock.title}</span>
            {block.isFixed && <span className="text-xs text-g
ray-400">📍 Fixed</span>}
            {block.priority !== "nice_to_do" && (
              <span className={`text-xs px-1.5 py-0.5 rounded
${PRIORITY_BADGES[block.priority] || ""}`}>
                {block.priority === "must_do" ? "⚡MUST DO" :
"SHOULD DO"}
              </span>
            )}
          </div>
          {block.aiReason && (
            <p className="text-xs text-gray-400 mt-0.5">{bloc
k.aiReason}</p>
          )}
          <div className="text-xs text-gray-500 mt-0.5">
            {block.durationMinutes} min • Energy: {block.ener
gyLevel === "high" ? "🔴" : block.energyLevel === "medium" ?
"🟡" : "🟢"} {block.energyLevel}
```

```
          </div>
        </div>
        {!block.isCompleted && !block.isSkipped && !block.isF
ixed && (
          <div className="flex items-center gap-1 flex-shrink
-0">
            <button onClick={onComplete} className="p-1 text-
green-600 hover:bg-green-50 rounded" title="Complete">
              <Check className="w-4 h-4" />
            </button>
            {block.type === "task" && (
              <button onClick={onStartFocus} className="p-1 t
ext-blue-600 hover:bg-blue-50 rounded" title="Start Focus">
                <Play className="w-4 h-4" />
              </button>
            )}
            <button onClick={onSkip} className="p-1 text-gray
-400 hover:bg-gray-100 rounded" title="Skip">
              <SkipForward className="w-4 h-4" />
            </button>
          </div>
        )}
      </div>
    </div>
  );
}
```

FEATURE 14: AI HABIT TRACKER

14.1 tRPC Router

Create file: `src/server/routers/habits.ts`

```
// src/server/routers/habits.ts
```

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";

export const habitsRouter = router({
  list: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input, ctx }) => {
      return prisma.habit.findMany({
        where: { userId: ctx.user.id, isActive: true, deleted
At: null },
        include: { completions: { where: { date: { gte: new D
ate(new Date().setDate(new Date().getDate() - 30)) } }, order
By: { date: "desc" } } },
        orderBy: { createdAt: "asc" },
      });
    }),

  create: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      name: z.string().min(1).max(100),
      icon: z.string().optional(),
      color: z.string().optional(),
      durationMinutes: z.number().min(5).max(480),
      frequency: z.enum(["daily", "weekdays", "custom"]).defa
ult("daily"),
      customDays: z.array(z.number().min(0).max(6)).optional
(),
      preferredTimeRange: z.object({ earliest: z.string(), la
test: z.string() }).optional(),
      priority: z.enum(["high", "medium", "low"]).default("me
dium"),
      isFlexible: z.boolean().default(true),
      isProtected: z.boolean().default(false),
    }))
```

```
    .mutation(async ({ input, ctx }) => {
      return prisma.habit.create({
        data: { ...input, userId: ctx.user.id },
      });
    }),

  update: protectedProcedure
    .input(z.object({
      id: z.string().uuid(),
      name: z.string().optional(),
      icon: z.string().optional(),
      color: z.string().optional(),
      durationMinutes: z.number().optional(),
      frequency: z.enum(["daily", "weekdays", "custom"]).opti
onal(),
      customDays: z.array(z.number()).optional(),
      preferredTimeRange: z.any().optional(),
      priority: z.enum(["high", "medium", "low"]).optional(),
      isFlexible: z.boolean().optional(),
      isProtected: z.boolean().optional(),
      isActive: z.boolean().optional(),
    }))
    .mutation(async ({ input }) => {
      const { id, ...data } = input;
      return prisma.habit.update({ where: { id }, data });
    }),

  complete: protectedProcedure
    .input(z.object({ habitId: z.string().uuid(), date: z.str
ing() }))
    .mutation(async ({ input, ctx }) => {
      const dateObj = new Date(input.date);

      // Create completion
      await prisma.habitCompletion.upsert({
        where: { habitId_date: { habitId: input.habitId, dat
```

```
e: dateObj } },
        create: { habitId: input.habitId, userId: ctx.user.i
d, date: dateObj },
        update: {},
      });

      // Update streak
      const habit = await prisma.habit.findUnique({
        where: { id: input.habitId },
        include: { completions: { orderBy: { date: "desc" },
take: 60 } },
      });

      if (habit) {
        // Calculate current streak
        let streak = 0;
        const today = new Date();
        for (let i = 0; i < 365; i++) {
          const checkDate = new Date(today);
          checkDate.setDate(checkDate.getDate() - i);
          const dateStr = checkDate.toISOString().split("T")
[0];
          const completed = habit.completions.some(
            (c) => c.date.toISOString().split("T")[0] === dat
eStr
          );
          if (completed) {
            streak++;
          } else if (i > 0) {
            break; // streak broken
          }
        }

        await prisma.habit.update({
          where: { id: input.habitId },
          data: {
```

```
            streakCurrent: streak,
            streakLongest: Math.max(streak, habit.streakLonge
st),
          },
        });
      }

      return { success: true };
    }),

  uncomplete: protectedProcedure
    .input(z.object({ habitId: z.string().uuid(), date: z.str
ing() }))
    .mutation(async ({ input }) => {
      await prisma.habitCompletion.delete({
        where: { habitId_date: { habitId: input.habitId, dat
e: new Date(input.date) } },
      }).catch(() => {}); // ignore if not found
      return { success: true };
    }),

  delete: protectedProcedure
    .input(z.object({ id: z.string().uuid() }))
    .mutation(async ({ input }) => {
      return prisma.habit.update({
        where: { id: input.id },
        data: { deletedAt: new Date() },
      });
    }),
});
```

FEATURE 9: AI FOCUS & PRODUCTIVITY COACH

9.1 tRPC Router

Create file: `src/server/routers/focus.ts`

```
// src/server/routers/focus.ts

import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const focusRouter = router({
  /**
   * Start a focus session
   */
  startSession: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      taskId: z.string().uuid().optional(),
      title: z.string(),
      plannedMinutes: z.number().min(5).max(480),
    }))
    .mutation(async ({ input, ctx }) => {
      // Check for existing active session
      const existing = await prisma.focusSession.findFirst({
        where: { userId: ctx.user.id, status: "active" },
      });
      if (existing) throw new Error("You already have an acti
ve focus session. End it first.");

      return prisma.focusSession.create({
        data: {
          userId: ctx.user.id,
          workspaceId: input.workspaceId,
          taskId: input.taskId,
          title: input.title,
          plannedMinutes: input.plannedMinutes,
          startedAt: new Date(),
```

```
          status: "active",
          notificationsPaused: true,
        },
      });
    }),

  /**
   * End a focus session
   */
  endSession: protectedProcedure
    .input(z.object({ sessionId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const session = await prisma.focusSession.findUnique({
where: { id: input.sessionId } });
      if (!session) throw new Error("Session not found");

      const endedAt = new Date();
      const actualMinutes = Math.round((endedAt.getTime() - s
ession.startedAt.getTime()) / 60000) - session.pausedMinutes;

      // Update session
      const updated = await prisma.focusSession.update({
        where: { id: input.sessionId },
        data: {
          endedAt,
          actualMinutes: Math.max(actualMinutes, 0),
          status: "completed",
          notificationsPaused: false,
        },
      });

      // Create time entry (Feature 21)
      if (session.taskId) {
        await prisma.timeEntry.create({
          data: {
            userId: ctx.user.id,
```

```
            workspaceId: session.workspaceId,
            taskId: session.taskId,
            focusSessionId: session.id,
            durationMinutes: Math.max(actualMinutes, 0),
            startedAt: session.startedAt,
            endedAt,
            source: "focus_session",
          },
        });
      }

      // Update daily productivity score
      const today = new Date().toISOString().split("T")[0];
      await prisma.productivityScore.upsert({
        where: { userId_date: { userId: ctx.user.id, date: ne
w Date(today) } },
        create: {
          userId: ctx.user.id,
          workspaceId: session.workspaceId,
          date: new Date(today),
          focusMinutes: Math.max(actualMinutes, 0),
        },
        update: {
          focusMinutes: { increment: Math.max(actualMinutes,
0) },
        },
      });

      return updated;
    }),

  /**
   * Pause/resume a focus session
   */
  togglePause: protectedProcedure
    .input(z.object({ sessionId: z.string().uuid() }))
```

```
    .mutation(async ({ input }) => {
      const session = await prisma.focusSession.findUnique({
where: { id: input.sessionId } });
      if (!session) throw new Error("Session not found");

      const newStatus = session.status === "active" ? "pause
d" : "active";
      return prisma.focusSession.update({
        where: { id: input.sessionId },
        data: { status: newStatus },
      });
    }),

  /**
   * Get active focus session (if any)
   */
  getActiveSession: protectedProcedure
    .query(async ({ ctx }) => {
      return prisma.focusSession.findFirst({
        where: { userId: ctx.user.id, status: { in: ["activ
e", "paused"] } },
      });
    }),

  /**
   * Get productivity scores for a date range (analytics)
   */
  getProductivityScores: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      startDate: z.string(),
      endDate: z.string(),
    }))
    .query(async ({ input, ctx }) => {
      return prisma.productivityScore.findMany({
        where: {
```

```
          userId: ctx.user.id,
          date: {
            gte: new Date(input.startDate),
            lte: new Date(input.endDate),
          },
        },
        orderBy: { date: "asc" },
      });
    }),

  /**
   * Get AI coaching insight
   */
  getCoachingInsight: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const thirtyDaysAgo = new Date();
      thirtyDaysAgo.setDate(thirtyDaysAgo.getDate() - 30);

      const scores = await prisma.productivityScore.findMany
({
        where: { userId: ctx.user.id, date: { gte: thirtyDays
Ago } },
        orderBy: { date: "asc" },
      });

      const sessions = await prisma.focusSession.findMany({
        where: { userId: ctx.user.id, status: "completed", st
artedAt: { gte: thirtyDaysAgo } },
      });

      const result = await aiService.generateText(
        `Analyze this user's productivity data and provide 2-
3 actionable coaching tips.

DAILY SCORES (last 30 days): ${JSON.stringify(scores.map(s =>
```

```
({
          date: s.date, focusMin: s.focusMinutes, meetingMin:
s.meetingMinutes,
          tasksCompleted: s.tasksCompleted, habitsCompleted:
s.habitsCompleted,
        }))}

FOCUS SESSIONS: ${sessions.length} total, avg ${sessions.leng
th > 0 ? Math.round(sessions.reduce((sum, s) => sum + (s.actu
alMinutes || 0), 0) / sessions.length) : 0} min each

Provide specific, actionable insights. Be encouraging but hon
est.`,
        {
          systemPrompt: "You are a productivity coach. Analyz
e the data and give specific, personalized tips. Be warm and
encouraging. Keep it to 2-3 short tips.",
          maxTokens: 500,
          temperature: 0.6,
        }
      );

      return { insight: result.text };
    }),
});
```

EDGE CASES FOR SPRINT 3

```
FEATURE 3 (Daily Planner):
1. No tasks and no meetings → Generate minimal plan with habi
ts + communication blocks + breaks
2. Too many tasks for one day → AI selects highest priority,
puts rest in "overflow" note
3. All-day meeting → Schedule tasks before/after meeting only
```

```
4. Calendar sync fails → Show warning, let user manually add
meetings
5. User drags a fixed meeting → Show "This meeting can't be m
oved. It's from your calendar."
6. Regenerate plan with completed blocks → Preserve completed
blocks, only reschedule remaining
7. Two meetings overlap → Show conflict warning with both eve
nts

FEATURE 14 (Habits):
1. Habit scheduled but day is full → AI moves to earliest ava
ilable slot, or suggests skipping
2. Streak broken → Show "Streak reset to 0. Previous best: 12
days. Start again!"
3. Complete future date → Block with "Can only complete today
or past dates"
4. Custom days has zero days → Validation error "Select at le
ast one day"

FEATURE 9 (Focus):
1. Start focus with existing active session → Error: "End you
r current session first"
2. Browser/tab closed during focus → On next visit, show "You
have an unfinished focus session" with [Resume] [End]
3. Focus session > 4 hours → Auto-pause with "You've been foc
using for 4 hours. Take a break!"
4. No productivity data yet → Show "Track a few focus session
s to see your insights"
```

SPRINT 3 VERIFICATION CHECKLIST

```
□ Navigate to /my-day → page loads
□ Click "Generate Plan" → AI creates daily plan with time blo
cks
□ Blocks show in chronological order with correct styling per
```

```
type
□ Click ✅ on a block → block marked complete, opacity reduce
d
□ Click ⏭️ on a block → block marked skipped
□ Click 🔄 Regenerate → new plan generated, completed blocks
preserved
□ Navigate to /habits → create a new habit "Deep Work 2h dail
y"
□ Complete the habit → streak shows "🔥 1 day"
□ Habit appears in daily plan timeline
□ Navigate to any task → click "Start Focus" → timer starts
□ Timer shows countdown, notifications indicator shows "pause
d"
□ End focus session → summary shown, time logged
□ View productivity analytics → charts show focus time trends
□ Click "Get Coaching" → AI provides personalized tips
□ Scheduling preferences saved and affect plan generation
```

Once all checks pass, proceed to Sprint 4.